CAPSTONE PROJECT
WEEK 1 DESIGN DOCUMENT

LAST-MILE ROUTE SEQUENCING AND RIDER PAYOUT ENGINE

Problem Statement: PS-035

Team ID: T007

Domain: Logistics – Last Mile

Proposed Language: Python 3.10+

TEAM MEMBERS

Arjun K Lapoji – 1AT24CD014

Arul Shakthi C – 1AT24CD015

Section: CSDS A

Department: Computer Science and Data Science

Atria Institute of Technology

Training Organisation: 10 SECONDS

Table of Contents

1. Project Introduction
2. Problem Statement
3. Project Objectives
4. Project Scope
5. Glossary of Terms
6. Functional Requirements
7. Non-Functional Requirements
8. System Workflow
9. UML Class Diagram
10. Parcel State Diagram
11. Business Rule Register
12. Edge Case Register
13. Assumptions and Open Questions
14. Preliminary Design Decision Log
15. Conclusion

1. Project Introduction

Last-mile delivery is the final stage of logistics, where parcels move from a local hub to their recipients. A delivery company must coordinate riders, vehicle limits, customer time windows, delivery outcomes and rider earnings. The Last-Mile Route Sequencing and Rider Payout Engine is a command-file-driven backend system designed to support these operations.

The engine will register riders, load parcels, assign parcels subject to capacity and timing constraints, sequence stops, calculate ETAs, record delivery events and calculate daily and weekly payouts. A graphical user interface is not required.

2. Problem Statement

A delivery hub may need to process hundreds of parcels each day. Manual parcel allocation and route planning can cause overloaded vehicles, inefficient routes, missed delivery windows and inconsistent rider payments. Payouts are complicated by per-parcel rates, distance pay, incentives, penalties, COD shortfalls, attendance bonuses, EV allowances and TDS.

The project proposes a structured backend engine that applies documented business rules consistently, validates inputs, handles boundary cases and produces outputs in the format required by the sample-data README.

3. Project Objectives

• Register riders with vehicle type, shift window, capacity and home hub.

• Load parcels with coordinates, weight, time window, priority and COD flag.

• Assign parcels without exceeding parcel-count or weight limits.

• Sequence delivery stops using nearest-insertion and calculate ETAs.

• Respect delivery time windows, shift limits and permitted overtime.

• Record delivered, failed and rescheduled outcomes and COD collected.

• Calculate daily earnings, incentives, penalties and COD shortfalls.

• Calculate weekly attendance bonus, EV allowance, TDS and net payout.

• Generate rider statements and machine-readable output files.

• Validate behaviour with sample data, edge-case tests and automated tests.

4. Project Scope

4.1 In Scope

• Rider registration and capacity validation.

• Parcel loading and validation.

• Parcel assignment and route sequencing.

• ETA, route-return and overtime calculations.

• Delivery event recording, rescheduling and RTO handling.

• Daily and weekly payout calculations.

• COD reconciliation and payout-freeze flagging.

• Command-file-driven input and output.

4.2 Out of Scope

• Customer-facing website or mobile application.

• Live GPS tracking unless separately specified.

• Payment-gateway integration or actual bank transfers.

• Live traffic prediction unless required by the sample-data specification.

• Authentication and production deployment.

The sample-data README is authoritative for command syntax, file columns and exact output formats.

5. Glossary of Terms

Term	Definition / relevance
Rider	Delivery worker who receives parcels and performs delivery stops.
Parcel	Package to be delivered to a recipient.
Hub	Central delivery location from which riders and parcels are dispatched.

Term	Definition / relevance
Last-mile delivery	Final movement of a parcel from a local hub to its recipient.
Route sequencing	Arranging delivery stops in an order that satisfies constraints.
Nearest-insertion heuristic	A route-building method that inserts a stop where it adds the least extra route distance.
ETA	Estimated Time of Arrival at a delivery stop.
Time window	Permitted time interval for delivering a parcel.
COD	Cash on Delivery; cash collected from the customer.
RTO	Return to Origin; parcel removed from the delivery pool after the reschedule limit.
BIKE / EV_BIKE / VAN	Vehicle categories with capacities defined in BR-01.
Odometer distance	Actual route distance recorded by the vehicle/rider.
Planned distance	Distance calculated for the planned route.
First-attempt success rate	Share of eligible deliveries successful on the first attempt; calculation needs clarification.
Incentive	Additional earnings awarded for meeting defined conditions.
Penalty	Deduction applied for a defined failure, late delivery or complaint.
TDS	Tax deducted at source; 1% when weekly gross exceeds ■5,000.
Overtime	Work beyond shift end, up to the 60-minute allowance.

6. Functional Requirements

ID	Requirement	Description	Input	Expected output
FR-01	Rider management	Register rider vehicle, shift, capacity and home hub.	Rider records	Validated rider records.
FR-02	Parcel management	Load parcel coordinates, weight, time window, priority and COD flag.	Parcel records	Validated parcel records.
FR-03	Assignment and routing	Assign parcels and sequence stops with ETAs.	Riders, parcels, hub	Routes, ETAs and unassigned parcels.
FR-04	Delivery events	Record DELIVERED, FAILED with reason, RESCHEDULED and COD collected.	Event files	Updated parcel states and event records.
FR-05	Daily payout	Calculate parcel pay, distance pay, incentives, penalties and COD handling.	Events, distances, complaints, COD deposits	Daily payout breakdown.
FR-06	Weekly payout	Calculate attendance, EV allowance, TDS, deductions and statements.	Daily records and weekly attendance	Weekly statement and net/frozen status.

7. Non-Functional Requirements

ID	Requirement	Description
NFR-01	Correctness	Apply confirmed business rules consistently.
NFR-02	Reliability	Invalid input must produce a clear validation outcome.
NFR-03	Performance	Process the supplied workload of approximately 300 parcels per day in practical time.
NFR-04	Maintainability	Separate responsibilities into understandable modules/classes.
NFR-05	Testability	Make each business rule independently testable.
NFR-06	Portability	Run using Python 3.10+ in a standard environment.
NFR-07	Auditability	Make route choices, payout components and flags traceable.
NFR-08	Input validation	Validate required fields, numeric ranges, dates, times and enumerated values.

8. System Workflow

The following workflow is conceptual. Exact command names and file formats must follow the sample-data README.

```
flowchart TD
 A[Load hub, rider and parcel files] --> B[Validate input]
 B --> C[Prepare riders]
 C --> D[Sort parcels by priority and window end]
 D --> E[Check capacity and shift feasibility]
 E --> F[Evaluate nearest-insertion positions]
 F --> G{Eligible rider and ETA?}
 G -- Yes --> H[Assign parcel and update route]
 G -- No --> I[Try next eligible rider]
 I --> G
 H --> J[Publish routes and ETAs]
 J --> K[Load delivery, odometer and COD events]
 K --> L[Update parcel outcomes]
 L --> M[Calculate daily payout]
 M --> N[Aggregate weekly payout]
 N --> O[Generate rider statements]
```

A parcel that cannot be assigned remains UNASSIGNED. Rescheduled parcels return to the next day's pool according to BR-11. A COD shortfall greater than ■2,000 freezes the weekly payout pending review.

9. UML Class Diagram

The proposed model separates rider and parcel data from route planning, delivery events and payout calculations.

```
classDiagram
 class Hub {
  +hub_id: str
  +name: str
  +latitude: float
  +longitude: float
 }
 class Rider {
  +rider_id: str
  +name: str
  +vehicle_type: VehicleType
  +shift_start: time
  +shift_end: time
  +parcel_capacity: int
  +weight_capacity_kg: float
  +home_hub_id: str
  +check_capacity()
 }
 class Parcel {
  +parcel_id: str
  +latitude: float
  +longitude: float
  +weight_kg: float
  +window_start: time
  +window_end: time
  +priority: Priority
  +cod_flag: bool
  +reschedule_count: int
  +status: ParcelStatus
  +upgrade_priority()
  +mark_rto()
 }
 class Route {
  +route_id: str
  +rider_id: str
  +hub_id: str
  +stops: list
  +planned_distance_km: float
  +estimated_return: time
  +calculate_eta()
  +insert_stop()
  +validate_shift()
 }
 class Delivery {
  +event_id: str
  +parcel_id: str
  +rider_id: str
  +outcome: DeliveryStatus
  +failure_reason: str
  +cod_collected: float
  +event_time: datetime
  +record_outcome()
 }
 class Payout {
  +rider_id: str
  +payout_date: date
  +parcel_pay: float
  +distance_pay: float
  +incentives: float
  +penalties: float
  +cod_shortfall: float
  +net_amount: float
  +calculate_daily()
  +calculate_weekly()
  +generate_statement()
 }
 Hub "1" --> "0..*" Rider : home hub
 Hub "1" --> "0..*" Route : dispatches
 Rider "1" --> "0..*" Route : operates
 Route "1" o-- "0..*" Parcel : ordered stops
 Rider "1" --> "0..*" Delivery : performs
 Parcel "1" --> "0..*" Delivery : has events
 Rider "1" --> "0..*" Payout : receives
```

A hub can be associated with many riders and routes. A rider can operate multiple routes over different days. A route contains ordered parcel stops. A parcel can have multiple delivery-event records across attempts. Payout records aggregate a rider's earnings and deductions.

10. Parcel State Diagram

This conceptual state machine describes the parcel lifecycle. The brief specifies rescheduling behaviour, but does not fully define all retry transitions; those require mentor confirmation.

```
stateDiagram-v2
 [*] --> REGISTERED
 REGISTERED --> UNASSIGNED : loaded and validated
 UNASSIGNED --> ASSIGNED : eligible rider selected
 ASSIGNED --> OUT_FOR_DELIVERY : route dispatched
 OUT_FOR_DELIVERY --> DELIVERED : delivery successful
 OUT_FOR_DELIVERY --> FAILED : attempt failed
 OUT_FOR_DELIVERY --> RESCHEDULED : reschedule event
 FAILED --> RESCHEDULED : reschedule approved
 RESCHEDULED --> UNASSIGNED : next-day pool / priority upgraded
 RESCHEDULED --> RTO : reschedule count reaches 2
 DELIVERED --> [*]
 RTO --> [*]
```

Confirm whether FAILED is a persistent parcel state or an attempt outcome, and how a failed parcel returns to the delivery pool.

11. Business Rule Register

ID / Name	Detailed rule and expected behaviour	Validation / notes
BR-01 Capacity	BIKE: 40 parcels / 25 kg. EV_BIKE: 40 / 25 kg. VAN: 120 / 300 kg. A rider must never exceed either limit.	Check parcel count and total weight after each proposed assignment. Exactly at capacity is permitted; above capacity is rejected.
BR-02 Assignment	Sort parcels by priority (EXPRESS first), then earliest time-window end. Assign to the eligible rider with lowest added route distance using nearest-insertion whose shift can accommodate the ETA. EXPRESS first stop must not exceed 09:30.	Try eligible riders in ascending added-distance order. If none is feasible, leave UNASSIGNED. Exactly 09:30 appears allowed by the word 'exceed'; confirm.
BR-03 Route time	Travel time is 4 minutes/km at 15 km/h, plus 3 minutes service per stop and 5 additional minutes for COD stops. ETA must fall inside the parcel time window.	Check both time-window boundaries. Confirm inclusivity and whether COD time is additional to normal service time.
BR-04 Shift	Route must return to hub before shift end. Overtime beyond shift end is allowed up to 60 minutes and paid at 1.5x distance pay for the overtime portion.	Handle routes exceeding the 60-minute overtime limit. Clarify the 'before shift end' wording versus allowed overtime.
BR-05 Per-parcel pay	STANDARD: ■18. EXPRESS: ■25. FAILED with NOT_HOME or WRONG_ADDRESS: ■10. FAILED with RIDER_FAULT: ■0.	Validate reason codes. Clarify payment treatment for other failure reasons and repeated attempts.
BR-06 Distance pay	■3.5/km on planned route. Actual odometer distance is used only if within 10% of planned; otherwise use planned distance and flag rider.	Exactly 10% above planned is within tolerance. Clarify whether tolerance is symmetric.
BR-07 Incentives	At least 30 delivered: ■100. At least 45 delivered: ■250, not cumulative. First-attempt success rate at least 95% adds ■50.	45+ receives ■250, not ■350. Confirm first-attempt numerator and denominator.
BR-08 Penalties	RIDER_FAULT: ■50 each. Late delivery outside time window: ■20 each. Customer complaint: ■100. Daily penalties capped at 25% of daily gross.	Calculate total penalties, then cap at 25% of daily gross. Clarify gross definition and duplicate events.
BR-09 COD	Deposit collected cash by 21:00 same day. Shortfall is deducted from payout. Shortfall greater than ■2,000 freezes weekly payout pending review.	Exactly ■2,000 does not trigger the strictly-greater-than freeze threshold. Clarify late deposit handling and release process.
BR-10 Weekly payout	■500 attendance bonus for 6 working days. EV_BIKE allowance ■30/day. TDS 1% if weekly gross exceeds ■5,000. Net rounded to nearest rupee.	Clarify working-day definition, deduction order, TDS base and rounding convention.
BR-11 Rescheduling	RESCHEDULED parcel returns to next day's pool with priority upgraded one level. A parcel rescheduled twice becomes RTO and leaves the pool.	Track reschedule count and priority. Confirm priority ladder and whether second reschedule immediately triggers RTO.

12. Edge Case Register

ID	Scenario	Expected result / clarification	Rule
EC-01	Rider has exactly 25.0 kg; proposed parcel is 0.1 kg.	Reject assignment: total would become 25.1 kg, exceeding the 25 kg limit.	BR-01
EC-02	Only feasible rider's EXPRESS first-stop ETA is exactly 09:30.	Appears eligible because the rule excludes a first stop that would exceed 09:30. Confirm boundary inclusivity.	BR-02
EC-03	Route returns exactly at shift end.	Ambiguous: 'before shift end' suggests not valid, while overtime is allowed beyond shift end. Obtain mentor clarification.	BR-04
EC-04	Actual odometer is exactly 10% above planned.	Accept actual distance as within the 10% tolerance, subject to mentor confirmation of tolerance interpretation.	BR-06
EC-05	30 deliveries, 29 successful on first attempt (96.7%).	Apply ■100 delivery slab and additional ■50 first-attempt incentive if the success-rate denominator confirms 96.7%.	BR-07
EC-06	Penalties exceed 25% of low daily gross.	Cap total daily penalties at 25% of that day's gross.	BR-08
EC-07	COD shortfall is exactly ■2,000.	Deduct ■2,000; do not freeze because the rule says greater than ■2,000.	BR-09
EC-08	Six working days; one day has only a FAILED attempt.	Attendance bonus depends on whether this counts as a working day. The brief does not define attendance; ask mentor.	BR-10

13. Assumptions and Open Questions

- Is an EXPRESS first-stop ETA of exactly 09:30 allowed?
- Is returning exactly at shift end valid, given the 'before shift end' wording and overtime allowance?
- Are parcel time-window start and end boundaries inclusive?
- Does a COD stop take 3 minutes normal service plus 5 additional minutes?
- How is the initial route created, and how are equal nearest-insertion costs broken?
- Are coordinates used for straight-line distance or road distance?
- How is first-attempt success rate calculated, including rescheduled or cancelled parcels?
- Does a day with only a FAILED attempt count as a working day?
- How is overtime distance identified and paid at 1.5x?
- Does the 10% odometer tolerance apply symmetrically above and below planned distance?
- What components constitute daily and weekly gross?
- Is COD shortfall deducted before or after the daily penalty cap?
- Is TDS calculated before or after incentives and other deductions?
- What is the exact priority ladder for a one-level upgrade?

• Does the second reschedule immediately create RTO?

• How is a frozen weekly payout released, and can undisputed earnings be paid?

• What rounding convention and intermediate precision should be used?

These are clarification questions, not new business rules. Send them to the mentor by replying on the original team assignment email thread.

14. Preliminary Design Decision Log

ID	Decision	Reason / impact
D-01	Use Python 3.10+ as the proposed language.	Readable syntax and suitable support for backend logic and testing.
D-02	Use a command-line/file-driven application.	The brief requires no UI or framework.
D-03	Separate Rider, Parcel, Route, Delivery and Payout responsibilities.	Improves domain clarity and testability.
D-04	Represent vehicle type, priority and status as controlled values.	Reduces invalid values.
D-05	Validate parcel count and total weight for each assignment.	BR-01 defines two independent limits.
D-06	Sort parcels by priority and then time-window end.	Follows BR-02.
D-07	Evaluate insertion positions by added route distance.	Implements nearest-insertion; tie-breaking remains to be confirmed.
D-08	Store planned and actual distance separately.	Supports BR-06 tolerance and flagging.
D-09	Store delivery attempts as events associated with parcels.	Supports failed attempts, rescheduling and first-attempt calculations.
D-10	Calculate uncapped penalties before applying the daily cap.	Makes BR-08 transparent and auditable.
D-11	Separate daily payout calculation from weekly aggregation.	Separates daily rules from weekly allowances and TDS.
D-12	Track COD collected, deposited and shortfall separately.	Supports reconciliation and payout freeze.
D-13	Map automated tests to BR and EC identifiers.	Provides traceable test coverage.
D-14	Keep unresolved boundary behaviour as open questions.	Avoids inventing business policy.
D-15	Follow the sample-data README for commands and output schemas.	Evaluation compares outputs field-for-field.

15. Conclusion

This design defines the system scope, domain vocabulary, functional and non-functional requirements, workflow, class structure, parcel lifecycle, 11 business rules and eight edge cases. It provides a structured foundation for implementation and testing in the following project weeks.

Before coding, the team should obtain mentor decisions on the open questions and confirm the exact command syntax and output schemas in the sample-data README. The design document should then be updated to reflect those confirmed interpretations.